AutoIA V1 Mathematical Foundations and Certification Architecture

A Self-Contained Specification Through Version 1.0.2

September 2026

Abstract

AutoIA is a research implementation of automatic validated interval analysis for existing engineering Python/NumPy programs. The central objective is to accept a deterministic engineering program, replace selected scalar and numerical operations by validated counterparts, and return output enclosures that contain the exact real semantics of every supported execution path for every input in a user-supplied interval box. The system is not based on naive line-by-line interval arithmetic alone. It combines explicit polynomials in the original uncertain inputs, interval remainders, shared auxiliary error generators, semantic certification of linear systems, path-aware execution, monotonicity and Bernstein range analysis, outward-rounded binary64 proof kernels, and prepared repeated-query certificate reuse.

This document is a mathematical specification of the current design through AutoIA version 1.0.2. Every mathematical symbol is defined before or at its first substantive use. The document separates *proposal calculations*, which may use ordinary floating point and may be inaccurate, from *proof calculations*, whose only purpose is to establish inclusion. In addition to the core polynomial, interval, generator, linear-solve, branching, and prepared-query machinery, the current V1 surface includes automatic configuration, exact closed-endpoint comparison metadata, proof-driven domain refinement, validated `root_scalar` and `fsolve` semantics, fixed-data multilinear `RegularGridInterpolator` semantics, parameter-dependent symmetric positive-definite matrix-family semantics for log determinants and inverse quadratic forms, parallel many-box CPU execution, optional non-trusted MPS/CUDA proposal acceleration with independent CPU verification, and an application-theorem interface. The document also states current non-claims, including the deliberate V1 fail-closed boundary at `scipy.linalg.eigh`.

Contents

1 Problem statement and certification target **5**

1.1 Three physical-input endpoint semantics 6

1.2 What is meant by program semantics 6

2 Notation used throughout the document **7**

3 Rigorous centering of the input box **8**

4 The representation-independent enclosure invariant **9**

5 The interval representation **9**

6 Point-coefficient polynomial plus interval remainder **10**
6.1 Multi-index monomial range on a symmetric box 10

7 Shared-generator polynomial representation **10**
7.1 Why fan-out cancellation improves 11
7.2 Closure under multiplication 11

8 Rigorous polynomial algebra **11**
8.1 Addition and subtraction 11
8.2 Truncated multiplication 12

9 Powers, reciprocal, and division **12**
9.1 Integer powers 12
9.2 Non-integer constant real powers 12
9.3 Reciprocal certificate 12

10 Trusted binary64 interval kernel and outward rounding **13**
10.1 Why a single outward adjustment after a long reduction is insufficient 13

11 Polynomial range analysis **13**
11.1 Termwise range 14
11.2 Derivative-based monotonicity 14
11.3 Bernstein enclosure 14
11.4 Monotonicity-reduced Bernstein 14
11.5 Portfolio principle 14

12 Validated elementary functions by Taylor composition **14**
12.1 Exponential 15
12.2 Logarithm 15
12.3 Sine and cosine 15
12.4 Error function and normal distribution functions 16

13 Semantic certification of parameter-dependent linear systems **16**
13.1 General polynomial solution predictor 16
13.2 Residual identity for an arbitrary predictor 17
13.3 Contraction theorem 17
13.4 Preconditioner portfolio 18

14 Validated symmetric positive-definite matrix families **18**
14.1 Constant proposal preconditioner 18
14.2 Rigorous eigenvalue enclosure 19
14.3 Spectral log-determinant bounds 20
14.4 Dependency-preserving matrix-log series 20
14.5 Inverse quadratic forms without entrywise inversion 21
14.6 Worked two-by-two example 21

15 Validated implicit nonlinear solves **22**
15.1 Dependency-bearing implicit predictor 22
15.2 Bracketed scalar `root_scalar` 22
15.3 Square-system `fsolve` 24
15.4 Derivative trust boundary 24

16 Generator-aware semantic linear solve **24**

17 Path-aware execution and uncertain control flow **25**

18 Exact closed-endpoint comparison metadata **26**

19 Proof-driven physical-domain refinement **26**

20 Fixed-data regular-grid multilinear interpolation **27**

21 Adaptive subdivision controller **28**

22 Automatic configuration as a portfolio of independent proofs **29**

23 Application-theorem interface **29**

24 Prepared sessions and repeated-query certification **30**
24.1 Exact repeated-box reuse 30
24.2 Restriction of a parent certificate to an interior subbox 30
24.3 Containment check for the actual centered query domain 31
24.4 Many-box execution and multicore CPU proof 32
24.5 MPS and CUDA proposal acceleration with CPU verification 32

25 Program-level soundness theorem **32**

26 What ordinary floating-point calculations are allowed to do **33**

27 V1 heterogeneous execution policy **34**

28 Important V1 limitations and explicit non-guarantees **34**

29 A compact end-to-end mathematical example **35**

30 Additional worked examples connecting the concepts **36**
30.1 Example A: polynomial truncation versus interval dependence loss 36
30.2 Example B: generator cancellation after approximation 36
30.3 Example C: path-aware absolute-value-like code 36
30.4 Example D: semantic linear solve 36
30.5 Example E: implicit solve followed by dependent arithmetic 36
30.6 Example F: interpolation across a grid boundary 37
30.7 Example G: independent theorem portfolio 37
30.8 Example H: correct fail-closed behavior for modal analysis 37
30.9 Example I: why an SPD matrix family is not an entrywise interval matrix 37
30.10 Example J: inverse quadratic form by a dependency-bearing series 37

30.11Example K: parallel boxes do not create a parallel-proof theorem 38
30.12Example L: GPU proposal versus CPU proof 38

31 Summary of the current mathematical architecture **38**

32 Transparent fitted exact-GP posterior semantics in Version 1.0.2 **39**

1 Problem statement and certification target

Let

$$\mathcal{P} : \mathbb{R}^m \to \mathbb{R}^r$$

denote the mathematical semantics of a supported deterministic engineering program. The symbol m denotes the number of uncertain scalar input parameters and r denotes the number of scalar outputs. The input vector is

$$\mathbf{p} = (p_1, \ldots, p_m)^T \in \mathbb{R}^m,$$

and the output vector is

$$\mathbf{y} = (y_1, \ldots, y_r)^T = \mathcal{P}(\mathbf{p}) \in \mathbb{R}^r.$$

The input-specification boundary requires distinguishing the physical interval the user intends from the finite binary64 interval on which AutoIA executes its proof. For uncertain input i, let

$$I_i^\star = [a_i, b_i] \subset \mathbb{R}, \qquad a_i \le b_i,$$

denote the *intended exact-real physical interval*. AutoIA's proof uses a finite binary64 interval

$$I_i = [\underline{p}_i, \overline{p}_i]$$

whose stored endpoints are interpreted as exact real numbers and which must satisfy

$$\boxed{I_i^\star \subseteq I_i.}$$

The intended physical box and the stored proof box are therefore

$$\mathcal{P}_{\text{in}}^\star = I_1^\star \times \cdots \times I_m^\star, \qquad \mathcal{P}_{\text{in}} = I_1 \times \cdots \times I_m,$$

with

$$\mathcal{P}_{\text{in}}^\star \subseteq \mathcal{P}_{\text{in}}.$$

When the user directly constructs `Interval(lo, hi)` from binary64 values, the intended and proof endpoints are those stored values themselves, so $I_i^\star = I_i$. When the user instead requests an exact-real enclosing constructor, the proof box may be widened by one adjacent binary64 value at an endpoint so that the exact intended endpoint is included.

Define the exact reachable sets over the intended and proof boxes by

$$\mathcal{Y}_{\text{intended}}^\star = \{\mathcal{P}(\mathbf{p}) : \mathbf{p} \in \mathcal{P}_{\text{in}}^\star\}, \qquad \mathcal{Y}_{\text{proof}}^\star = \{\mathcal{P}(\mathbf{p}) : \mathbf{p} \in \mathcal{P}_{\text{in}}\}.$$

Because $\mathcal{P}_{\text{in}}^\star \subseteq \mathcal{P}_{\text{in}}$,

$$\mathcal{Y}_{\text{intended}}^\star \subseteq \mathcal{Y}_{\text{proof}}^\star.$$

AutoIA certifies the stronger proof-box statement. It does not promise to compute the exact interval hull. For every scalar output y_j, AutoIA returns an interval

$$Y_j = [\underline{y}_j, \overline{y}_j]$$

such that

$$\boxed{y_j(\mathbf{p}) \in Y_j \quad \text{for every } \mathbf{p} \in \mathcal{P}_{\text{in}}.}$$

The same interval therefore also contains every output generated by the intended physical box $\mathcal{P}_{\text{in}}^\star$. For a piecewise/path-aware result, the system may instead return a finite union of certified branch enclosures. The reported global lower and upper bounds are then the interval hull of that finite union.

1.1 Three physical-input endpoint semantics

AutoIA 1.0.2 exposes three deliberately distinct input constructors.

Direct binary64 endpoints. The existing form

```
Interval(0.1, 0.3)
```

continues to mean exactly the two binary64 values that Python supplied. No claim is made that the source spelling `0.1` is the exact rational number 1/10. This backward-compatible interpretation is called `binary64-exact` input semantics.

Exact-real enclosing endpoints. The form

```
Interval.enclosing("0.1", "0.3")
```

interprets the strings as exact decimal real numbers and constructs finite binary64 proof endpoints satisfying

$$\underline{p}_i \le a_i, \qquad b_i \le \overline{p}_i.$$

Exact integers, `fractions.Fraction`, and `decimal.Decimal` values are also accepted. Ordinary Python or NumPy floating-point values are rejected by this constructor because their pre-rounding mathematical provenance has already been lost.

Internally, each exact decimal or rational endpoint is represented as an exact rational number. Conversion to binary64 is used only as a proposal. AutoIA compares the proposed binary64 value back to the exact rational value and, when necessary, steps to the adjacent binary64 value in the required outward direction. The final inequality is then checked exactly. If finite binary64 containment cannot be established, construction fails closed.

One-ulp padding of stored binary64 endpoints. The convenience form

```
Interval.outward(0.1, 0.3)
```

moves the supplied lower stored binary64 value one adjacent representable value toward $-\infty$ and the upper value one adjacent representable value toward $+\infty$. This is useful conservative padding, but it is *not* a proof of the provenance of an arbitrarily computed endpoint. In particular, one ulp cannot in general repair an unknown chain of earlier floating-point rounding errors.

The result metadata records the selected input semantics, the supplied endpoint specification, and the stored proof interval. This makes the exact theorem being certified inspectable rather than implicit.

1.2 What is meant by program semantics

For generic supported scalar arithmetic, stored binary64 constants in the user program are interpreted as exact real constants equal to their stored values. Recognized mathematical primitives are interpreted according to their documented real mathematical semantics. Examples include exact real exp, log, sin, cos, erf, normal PDF/CDF, and the exact solution of a recognized linear system $Ax = b$. The semantic linear-solve primitive therefore certifies the mathematical solution of the represented matrix family, not the floating-point rounding behavior of NumPy's implementation of `linalg.solve`.

Ordinary numeric constants in engineering code are intentionally *not* reinterpreted by the 1.0.2 input-specification patch. For example, the source literal `0.37` is first converted by Python to a

binary64 value, and AutoIA then treats that stored value as an exact real program constant. A coefficient whose physical uncertainty matters should be exposed as an uncertain input. A quantity that is genuinely intended as an exact mathematical rational can often be written using exactly representable integers and supported exact-real operations, for example `x/10` rather than requiring every decimal literal in a legacy code to be replaced by an AutoIA-specific object. Supported mathematical primitives such as exp are certified according to their stated real semantics and do not rely on trusting a bare platform approximation of a named transcendental constant.

If the user manually codes an iterative or approximate algorithm that is not a recognized semantic primitive, the default goal is to certify the supported program semantics as written. AutoIA does not silently replace an arbitrary legacy algorithm by a different mathematical problem merely because the latter looks like the likely engineering intent.

2 Notation used throughout the document

This section defines the notation that appears later.

Symbol	Definition
m	Number of uncertain scalar physical inputs.
r	Number of scalar outputs.
$\mathbf{p}$	Physical input vector $(p_1, \dots, p_m)^T$.
$I_i^\star$	Intended exact-real physical interval for uncertain input i.
I_i	Stored finite binary64 proof interval for uncertain input i, with $I_i^\star \subseteq I_i$.
$\mathcal{P}_{\text{in}}^\star$	Intended physical Cartesian input box $\prod_i I_i^\star$.
$\mathcal{P}_{\text{in}}$	Stored binary64 proof box $\prod_i I_i$; AutoIA certifies on this box.
$p_{c,i}$	Stored binary64 center chosen for input interval I_i.
q_i	Centered physical coordinate $q_i = p_i - p_{c,i}$.
h_i	Rigorous nonnegative half-width satisfying $q_i \in [-h_i, h_i]$.
$\mathbf{q}$	Centered physical vector $(q_1, \dots, q_m)^T$.
$\mathcal{Q}$	Symmetric centered box $\prod_{i=1}^m [-h_i, h_i]$.
α	Multi-index $(\alpha_1, \dots, \alpha_m)$ with nonnegative integer components.
$\lvert\alpha\rvert$	Total degree $\alpha_1 + \cdots + \alpha_m$.
$\mathbf{q}^\alpha$	Monomial $\prod_{i=1}^m q_i^{\alpha_i}$.
e_i	Unit multi-index whose i-th component is one and all others are zero.
$P(\mathbf{q})$	Stored point-coefficient polynomial in the original centered physical inputs.
R	Interval remainder, usually written $[\underline{R}, \overline{R}]$.
s	Number of active auxiliary generator variables.
$\boldsymbol{\varepsilon}$	Auxiliary vector $(\varepsilon_1, \dots, \varepsilon_s)^T$ with each $\varepsilon_j \in [-1, 1]$.
G	Generator coefficient matrix. For a scalar, this reduces to a row/vector of coefficients.
Γ_E	Concretization/interpretation of a validated representation E.
$A(\mathbf{q})$	Parameter-dependent matrix in a semantic linear system.
$b(\mathbf{q})$	Parameter-dependent right-hand side in a semantic linear system.
$X(\mathbf{q})$	Stored polynomial predictor for a linear-system solution.
C	Ordinary floating-point point preconditioner used in a linear-system certificate.
B	Nonnegative matrix satisfying a rigorous componentwise defect bound.

Symbol	Definition
c	Nonnegative vector satisfying a rigorous preconditioned residual bound.
ρ	Rigorous upper bound on $\|B\|_\infty$.
$\|B\|_\infty$	Maximum absolute row-sum norm $\max_i \sum_j \|B_{ij}\|$; since $B \geq 0$, this is $\max_i \sum_j B_{ij}$.
hull(S)	Smallest interval containing a scalar set S, or componentwise interval hull for a vector set.
down(x)	A binary64 lower endpoint known to be no greater than the exact real quantity x.
up(x)	A binary64 upper endpoint known to be no less than the exact real quantity x.

Componentwise vector/matrix inequalities such as

$$|v| \leq c$$

mean $|v_i| \leq c_i$ for every component i. Similarly,

$$|M| \leq B$$

means $|M_{ij}| \leq B_{ij}$ for every entry.

3 Rigorous centering of the input box

Polynomial representations use centered coordinates. For each user interval

$$I_i = [\underline{p}_i, \overline{p}_i],$$

AutoIA chooses a finite binary64 proposal center $p_{c,i}$. Conceptually this is near the arithmetic midpoint, but the proof does not require the chosen center to be the exact midpoint. The trusted backend then rigorously encloses

$$I_i - p_{c,i}$$

using outward-rounded interval subtraction. If the resulting interval is $[\ell_i, u_i]$, AutoIA defines

$$h_i = \max(|\ell_i|, |u_i|).$$

Therefore

$$I_i \subseteq [p_{c,i} - h_i, p_{c,i} + h_i],$$

and hence every physical input in I_i admits the centered coordinate

$$q_i = p_i - p_{c,i} \in [-h_i, h_i].$$

The complete centered domain is

$$\mathcal{Q} = \prod_{i=1}^{m} [-h_i, h_i].$$

The symmetric centered domain can be slightly larger than the original user interval because outward rounding may enlarge a boundary by one or more ulps. This enlargement is sound but matters near function-domain boundaries. For example, a user interval whose exact lower endpoint is zero may produce a centered polynomial domain whose rigorous reconstructed lower endpoint is a tiny negative number. AutoIA must not silently clip that negative number when certifying $\sqrt{x}$ or $\log(x)$; the current implementation fails closed in such a case unless a representation/path/domain refinement proves the required real domain.

4 The representation-independent enclosure invariant

Every validated scalar value is associated with a representation object E and a concretization map

$$\Gamma_E(\mathbf{q}) \subseteq \mathbb{R}.$$

The fundamental invariant is

$$\boxed{z(\mathbf{q}) \in \Gamma_E(\mathbf{q}) \quad \text{for all } \mathbf{q} \in \mathcal{Q},}$$

where z is the exact real semantic quantity represented by E.

Suppose an exact mathematical operation is denoted

$$z = \mathrm{op}(z_1, \ldots, z_k).$$

A validated implementation $\mathrm{op}^{\#}$ is sound when

$$\boxed{\Gamma_{\mathrm{op}^{\#}(E_1,\ldots,E_k)}(\mathbf{q}) \supseteq \mathrm{op}\big(\Gamma_{E_1}(\mathbf{q}), \ldots, \Gamma_{E_k}(\mathbf{q})\big)}$$

for every $\mathbf{q} \in \mathcal{Q}$, with the right-hand side interpreted as the set of all values of the exact operation on arguments chosen from the input sets.

Representation conversion is similarly required to be inclusion-preserving. If E_A is converted to E_B, then

$$\Gamma_{E_A}(\mathbf{q}) \subseteq \Gamma_{E_B}(\mathbf{q})$$

for every $\mathbf{q}$. Promotion priorities in the software influence strategy only; they are not themselves part of the proof.

5 The interval representation

The simplest scalar representation is a single interval

$$E = I = [\underline{z}, \overline{z}].$$

Its concretization is independent of $\mathbf{q}$:

$$\Gamma_E(\mathbf{q}) = I.$$

This representation forgets explicit dependence on the original physical inputs, but it is robust and useful as a fallback or for calculations where interval semantics are already natural.

The natural interval extension of an exact operation is used when supported. For example, if $X = [\underline{x}, \overline{x}]$ and $Y = [\underline{y}, \overline{y}]$, then interval addition returns an outward rounded enclosure of

$$[\underline{x} + \underline{y}, \overline{x} + \overline{y}].$$

Interval multiplication encloses all four endpoint products. Division requires a denominator interval separated from zero.

6 Point-coefficient polynomial plus interval remainder

The principal dependency-preserving representation is

$$\boxed{z(\mathbf{q}) \in P(\mathbf{q}) + R,}$$

where

$$P(\mathbf{q}) = \sum_{\alpha \in \mathcal{A}} c_\alpha \mathbf{q}^\alpha$$

is a sparse polynomial with stored binary64 point coefficients c_α, $\mathcal{A}$ is a finite set of multi-indices, and

$$R = [\underline{R}, \overline{R}]$$

is an interval remainder. The concretization is

$$\Gamma_E(\mathbf{q}) = P(\mathbf{q}) + R = \{P(\mathbf{q}) + r : r \in R\}.$$

The stored coefficient c_α is interpreted as an exact real number equal to its binary64 value. When a coefficient is produced by floating-point proposal arithmetic, AutoIA separately computes a rigorous interval containing the exact coefficient that the mathematical operation would produce, subtracts the stored point coefficient, and ranges that coefficient error over the input domain into R. This separation allows fast ordinary floating-point proposals while keeping the final enclosure rigorous.

6.1 Multi-index monomial range on a symmetric box

For

$$\alpha = (\alpha_1, \ldots, \alpha_m),$$

define

$$H_\alpha = \prod_{i=1}^{m} h_i^{\alpha_i}.$$

The trusted implementation computes an upward-rounded upper bound for this nonnegative product. On the symmetric box $\mathcal{Q}$, the monomial range is

$$\mathbf{q}^\alpha \in \begin{cases} [0, H_\alpha], & \text{if every } \alpha_i \text{ is even,} \\ [-H_\alpha, H_\alpha], & \text{otherwise.} \end{cases}$$

This parity-aware rule is sharper than blindly using $[-H_\alpha, H_\alpha]$ for every monomial.

7 Shared-generator polynomial representation

The polynomial-plus-remainder representation loses correlation when the same unstructured error is reused. AutoIA therefore also supports

$$\boxed{z(\mathbf{q}, \boldsymbol{\varepsilon}) \in P(\mathbf{q}) + \sum_{j=1}^{s} g_j \varepsilon_j + R, \qquad \varepsilon_j \in [-1, 1].}$$

For a vector of r outputs this is

$$\boxed{\mathbf{z}(\mathbf{q}, \boldsymbol{\varepsilon}) \in \mathbf{P}(\mathbf{q}) + G\boldsymbol{\varepsilon} + R,}$$

where $G \in \mathbb{R}^{r \times s}$ and R denotes a componentwise interval box.

The variables ε_j are *auxiliary* uncertainty variables. They do not represent additional physical uncertain inputs. Their purpose is to preserve provenance of approximation/certification error. Two coefficients bearing the same generator identifier refer to the same ε_j; distinct identifiers are treated as independent coordinates in $[-1, 1]^s$.

This distinction prevents an unsound shortcut. If two unrelated calculations happen to produce the same numerical error radius, AutoIA does not infer that the errors are identical. Shared provenance is created only by copying/reusing a value or by a semantic operation that rigorously carries the same generator through its output predictor.

7.1 Why fan-out cancellation improves

Suppose

$$y = P(\mathbf{q}) + g\varepsilon_1.$$

If the same represented value is used twice, then

$$y - y = P - P + g\varepsilon_1 - g\varepsilon_1 = 0.$$

A pure interval remainder would instead behave like two independent copies of the same interval and could fail to cancel.

7.2 Closure under multiplication

The representation is affine in $\boldsymbol{\varepsilon}$, so it is not algebraically closed under multiplication. If

$$x = P_x + G_x\boldsymbol{\varepsilon} + R_x, \qquad y = P_y + G_y\boldsymbol{\varepsilon} + R_y,$$

then products such as

$$q_i\varepsilon_j \quad \text{and} \quad \varepsilon_i\varepsilon_j$$

appear. The current representation keeps the constant-in-$\mathbf{q}$, first-order in-$\boldsymbol{\varepsilon}$ contribution explicitly and rigorously ranges terms outside the representation family into fresh bounded error. Soundness is therefore retained without pretending that $P(\mathbf{q}) + G\boldsymbol{\varepsilon} + R$ is closed under arbitrary nonlinear operations.

8 Rigorous polynomial algebra

8.1 Addition and subtraction

Let

$$x \in P_x + R_x, \qquad y \in P_y + R_y.$$

The exact algebraic sum is

$$x + y \in (P_x + P_y) + (R_x + R_y).$$

AutoIA computes point coefficient proposals for $P_x + P_y$, rigorously encloses coefficient-rounding errors, and adds their ranged effect to the remainder. Subtraction is addition of the negated representation.

8.2 Truncated multiplication

Let the retained maximum polynomial order be K. The exact polynomial product is

$$P_x P_y = \sum_{\gamma} C_\gamma \mathbf{q}^\gamma, \qquad C_\gamma = \sum_{\alpha+\beta=\gamma} c_\alpha d_\beta.$$

For every γ satisfying $|\gamma| \le K$, AutoIA stores a point proposal $\widehat{C}_\gamma$. It rigorously encloses the exact coefficient C_γ and returns the difference

$$C_\gamma - \widehat{C}_\gamma$$

as a coefficient-error interval. That error is multiplied by a rigorous monomial range and accumulated into the remainder.

For discarded terms with $|\gamma| > K$, the full exact coefficient interval is ranged over $\mathcal{Q}$ and added to the remainder. The user-selected order K therefore changes sharpness and cost but not the inclusion contract.

The complete product also contains interactions with existing remainders. These are enclosed by ordinary interval multiplication using rigorous ranges of the retained polynomial parts.

9 Powers, reciprocal, and division

9.1 Integer powers

For $n \in \mathbb{Z}$, positive powers are constructed by validated multiplication. For $n < 0$,

$$x^n = \frac{1}{x^{-n}},$$

so AutoIA must prove that zero is excluded from the denominator range. Negative bases are valid for integer exponents.

9.2 Non-integer constant real powers

For a finite constant binary64 exponent $p \in \mathbb{R} \setminus \mathbb{Z}$, the current real branch requires the certified base range to be strictly positive. AutoIA may decompose

$$p = n + r,$$

where $n \in \mathbb{Z}$ and $|r| \le 1/2$, then use integer powers together with $\sqrt{\cdot}$ for half powers or the identity

$$x^r = \exp(r \log x)$$

for the remaining positive-real case. An uncertain exponent is outside the current supported contract.

9.3 Reciprocal certificate

Suppose a represented denominator y has a rigorous range

$$Y = [\underline{Y}, \overline{Y}]$$

with

$$0 \notin Y.$$

Let Q be any stored polynomial proposal for $1/y$. The exact identity

$$\frac{1}{y} - Q = \frac{1 - yQ}{y}$$

allows AutoIA to bound the residual $1 - yQ$, divide by the denominator interval Y, and obtain a certified reciprocal. The quality of Q affects only the remainder width.

10 Trusted binary64 interval kernel and outward rounding

Machine rigor requires every proof-critical floating-point operation to enclose its exact real counterpart. AutoIA therefore distinguishes two classes of calculation.

1. **Proposal calculations.** These may use ordinary round-to-nearest binary64, NumPy linear algebra, heuristic scores, or approximate inverses. Their outputs are never trusted merely because they were computed.

2. **Proof calculations.** These establish interval inclusion and are performed by the audited trusted backend or by operations reducible to trusted interval primitives.

   For a scalar operation $z = x \circ y$, the trusted backend returns binary64 endpoints

   $$[\underline{z}, \overline{z}]$$

known to contain the exact real value. On the reference backend this is done by round-to-nearest arithmetic followed by `nextafter` toward $-\infty$ for the lower endpoint and $+\infty$ for the upper endpoint.

The backend construction audits the binary64 environment. The audit checks, among other things, the binary64 format, NumPy `float64` size, round-to-nearest behavior, adjacent `nextafter` values, gradual underflow/subnormals, and required vectorized arithmetic behavior. If the environment does not satisfy the assumptions, AutoIA fails closed.

10.1 Why a single outward adjustment after a long reduction is insufficient

A long dot product

$$s = \sum_{i=1}^{n} a_i b_i$$

contains rounding error in every multiplication and every addition. Merely computing an ordinary BLAS result and moving it outward by one ulp does not in general enclose the exact real dot product.

The Step-9/9b trusted fast path instead encloses every product and combines those product intervals through a pairwise interval-addition tree. If each leaf contains its exact product and each internal node contains the exact sum of its two child sets, induction proves that the root interval contains the exact dot product. Vectorization changes execution speed, not the mathematical proof.

The same idea is used for point-by-interval and interval-by-interval reductions, and many independent dot-product trees may be batched simultaneously across matrix rows/columns.

11 Polynomial range analysis

A validated symbolic representation is useful only if it can be converted to a scalar interval bound. AutoIA uses several sound range engines.

11.1 Termwise range

For

$$P(\mathbf{q}) = \sum_{\alpha} c_\alpha \mathbf{q}^\alpha,$$

each monomial is enclosed separately and the resulting intervals are summed. This method is inexpensive and robust, but it may lose cancellation between terms.

11.2 Derivative-based monotonicity

For coordinate q_i, AutoIA constructs the polynomial derivative

$$\frac{\partial P}{\partial q_i} = \sum_{\alpha:\alpha_i>0} \alpha_i c_\alpha \mathbf{q}^{\alpha-e_i}.$$

If a rigorous range proves

$$\frac{\partial P}{\partial q_i} \geq 0$$

throughout the box, then P is nondecreasing in q_i; extrema occur at the appropriate endpoint of that coordinate. If the derivative is rigorously nonpositive, the opposite endpoint is used. Monotone directions can therefore be fixed before ranging the unresolved directions.

11.3 Bernstein enclosure

After mapping the active box to a unit box, the polynomial can be expressed in a multivariate Bernstein basis. The range of a polynomial over the unit box is contained in the interval hull of its Bernstein coefficients. AutoIA computes an outward-rounded version of this transformation. Because the number of Bernstein coefficients grows as a product of per-variable degrees, the method is used selectively.

11.4 Monotonicity-reduced Bernstein

If some coordinates are already proved monotone, those coordinates are fixed at appropriate endpoints and Bernstein conversion is applied only to unresolved directions. This can greatly reduce coefficient count.

11.5 Portfolio principle

If two independent rigorous methods produce intervals I_1 and I_2 that both contain the same quantity, their intersection also contains that quantity, provided the intersection is nonempty. AutoIA therefore uses range-method portfolios to improve sharpness without making correctness depend on which method happens to be tighter.

12 Validated elementary functions by Taylor composition

For a scalar represented argument x, AutoIA first obtains a rigorous argument interval

$$X = [\underline{x}, \overline{x}].$$

Choose a stored binary64 expansion center $c \in X$ and write

$$\delta = x - c.$$

For a smooth function f, a degree-K Taylor representation is

$$f(x) = \sum_{n=0}^{K} a_n \delta^n + R_{K+1}, \qquad a_n = \frac{f^{(n)}(c)}{n!}.$$

The coefficient a_n is itself represented by a rigorous interval. AutoIA chooses a point proposal $\widehat{a}_n$, stores it in the polynomial, and propagates the difference between the rigorous coefficient interval and $\widehat{a}_n$ as certified error. A rigorous analytic bound on the omitted Taylor tail R_{K+1} is then added.

This generic composition mechanism is used for exp, log, $\sqrt{\cdot}$, sin, cos, erf, the standard normal PDF, and the standard normal CDF, with function-specific coefficient recurrences and tail bounds.

12.1 Exponential

For sufficiently small $|r|$,

$$e^r = \sum_{k=0}^{N} \frac{r^k}{k!} + R_N.$$

If $|r| \le a < 1$, one convenient rigorous tail bound is

$$|R_N| \le \frac{a^{N+1}}{(N+1)!(1-a)}.$$

The trusted scalar kernel repeatedly halves a rigorous argument interval until it is small enough for the series, then reconstructs the result by trusted squaring.

12.2 Logarithm

For $x > 0$, binary exponent extraction produces

$$x = 2^k m,$$

with a scaled mantissa m. Then

$$\log x = k \log 2 + \log m.$$

Using

$$z = \frac{m-1}{m+1},$$

we have

$$\log m = 2 \operatorname{atanh}(z),$$

which is evaluated by a rigorously bounded odd series. The constant $\log 2$ is itself generated rigorously rather than trusted from a platform library.

12.3 Sine and cosine

A rigorous interval for π is generated from Machin's formula. An ordinary floating-point integer reduction index may be proposed, but the proof forms the reduced argument with the rigorous π interval. Taylor sine/cosine bounds are used when the reduced interval is small enough; otherwise a safe range such as $[-1, 1]$ is available.

12.4 Error function and normal distribution functions

The error function is

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\, dt.$$

Moderate positive arguments use a bounded alternating Maclaurin series. Larger arguments use the complementary error function and an integration-by-parts asymptotic bracket. Odd symmetry handles negative arguments.

The standard normal density and CDF are

$$\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2},$$

$$\Phi(x) = \frac{1}{2}\left(1 + \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right)\right).$$

Their polynomial Taylor coefficients are generated using differential recurrences, including

$$\phi'(x) + x\phi(x) = 0$$

and

$$\Phi'(x) = \phi(x).$$

A normal distribution with constant mean μ and standard deviation $\sigma > 0$ is handled using

$$z = \frac{x - \mu}{\sigma}, \qquad f_X(x) = \frac{\phi(z)}{\sigma}, \qquad F_X(x) = \Phi(z).$$

13 Semantic certification of parameter-dependent linear systems

Consider a represented family of linear systems

$$A(\mathbf{q})x(\mathbf{q}) = b(\mathbf{q}),$$

where $A(\mathbf{q}) \in \mathbb{R}^{n\times n}$, $x(\mathbf{q}) \in \mathbb{R}^n$, and $b(\mathbf{q}) \in \mathbb{R}^n$. The matrix and right-hand side may have retained polynomial coefficients of arbitrary supported order plus rigorous remainders.

13.1 General polynomial solution predictor

Write

$$A(\mathbf{q}) \approx \sum_{\beta} A_\beta \mathbf{q}^\beta, \qquad b(\mathbf{q}) \approx \sum_{\beta} b_\beta \mathbf{q}^\beta,$$

and propose

$$X(\mathbf{q}) = \sum_{|\alpha|\le K} x_\alpha \mathbf{q}^\alpha.$$

Formal coefficient matching gives the recurrence

$$\boxed{A_0 x_\alpha = b_\alpha - \sum_{\substack{\beta\neq 0 \\ \beta\le\alpha}} A_\beta x_{\alpha-\beta}.}$$

Here $\beta \le \alpha$ means $\beta_i \le \alpha_i$ for every coordinate. The recurrence is only a predictor generator. AutoIA may use an ordinary floating-point solve, a different reference matrix, a family-hull midpoint, or another finite proposal.

13.2 Residual identity for an arbitrary predictor

Let X be whatever predictor is actually stored. Define the true solution error

$$e = x - X.$$

Define the residual

$$r = b - AX.$$

Because $Ax = b$,

$$A(X + e) = b,$$

so

$$\boxed{Ae = r.}$$

This identity is exact and does not depend on how X was obtained.

Choose any finite point matrix C as a preconditioner. Then

$$CAe = Cr.$$

Add and subtract e:

$$e - (I - CA)e = Cr,$$

and hence

$$\boxed{e = Cr + (I - CA)e.}$$

Suppose trusted calculations establish a nonnegative vector c and nonnegative matrix B satisfying, for every admissible input,

$$|Cr| \le c,$$
$$|I - CA| \le B.$$

Taking componentwise absolute values gives

$$|e| \le c + B|e|.$$

13.3 Contraction theorem

Define

$$\rho = \|B\|_\infty = \max_i \sum_j B_{ij}.$$

If a rigorous upper bound proves

$$\boxed{\rho < 1,}$$

then the family is nonsingular throughout the represented domain and

$$\boxed{|e| \le (I - B)^{-1} c.}$$

A simpler scalar consequence is obtained by defining

$$\eta = \|c\|_\infty, \qquad \delta = 1 - \rho > 0.$$

Then

$$\|e\|_\infty \le \frac{\eta}{\delta}.$$

The implementation computes an outward-rounded upper bound for this ratio and may subsequently refine the componentwise correction by the monotone iteration

$$d^{(k+1)} = \min\left(d^{(k)},\ \mathrm{up}(c + Bd^{(k)})\right),$$

starting from a safe scalar vector bound.

13.4 Preconditioner portfolio

The center matrix inverse is a useful proposal but can be a poor contraction preconditioner. AutoIA also constructs a rigorous interval hull of the matrix family, chooses a finite representative such as its componentwise midpoint, and proposes the inverse of that representative. Every candidate is checked against the complete matrix family. Therefore a midpoint that is not physically realizable still yields a sound certificate if the final defect bound passes.

14 Validated symmetric positive-definite matrix families

Many engineering calculations use a symmetric matrix whose entries depend on common uncertain inputs. Let

$$A(\mathbf{p}) \in \mathbb{R}^{n\times n}, \qquad \mathbf{p} \in \mathcal{P}_{\text{in}},$$

where n is the matrix dimension. Assume the matrix is intended to satisfy

$$A(\mathbf{p}) = A(\mathbf{p})^T$$

for every admissible $\mathbf{p}$. The notation

$$A(\mathbf{p}) \succ 0$$

means that the matrix is symmetric positive definite (SPD), namely

$$\mathbf{x}^T A(\mathbf{p})\mathbf{x} > 0 \quad \text{for every nonzero } \mathbf{x} \in \mathbb{R}^n.$$

This structure appears in covariance matrices, structural stiffness and mass matrices, Fisher information matrices, normal-equation matrices, Gaussian likelihoods, Kalman filters, and many other applications.

A naive entrywise interval matrix can be much larger than the actual matrix family because every entry is then allowed to vary independently. AutoIA therefore keeps the original shared physical dependence in the scalar matrix entries and applies a structured preconditioning theorem.

14.1 Constant proposal preconditioner

Let $\mathbf{p}_c$ be a nominal point, usually the center of the physical input box, and let

$$A_c = A(\mathbf{p}_c)$$

be an ordinary binary64 proposal matrix. AutoIA attempts an ordinary numerical Cholesky factorization

$$A_c \approx LL^T,$$

where L is lower triangular with positive diagonal. This factorization is *not* trusted as a proof. The stored binary64 entries of L are simply interpreted as exact real constants in all subsequent validated calculations. If the nominal factorization is inconvenient, another nonsingular positive-diagonal scaling may be used; proof correctness does not depend on L being an exact factor of A_c.

Define the preconditioned matrix family

$$B(\mathbf{p}) = L^{-1}A(\mathbf{p})L^{-T}.$$

Because multiplication by a fixed nonsingular matrix is a congruence transformation,

$$A(\mathbf{p}) \succ 0 \quad \Longleftrightarrow \quad B(\mathbf{p}) \succ 0.$$

Thus the proof problem can be moved to a family that is normally much closer to the identity.

14.2 Rigorous eigenvalue enclosure

AutoIA first proves that the represented matrix family is structurally symmetric. Equal numerical interval endpoints alone do not prove that two uncertain off-diagonal entries are the same expression; the symmetry proof must retain the common dependency or prove their exact difference is zero.

For each row i, let

$$B_{ii}(\mathbf{p}) \in [d_i^-, d_i^+]$$

be a rigorous enclosure of the diagonal entry and let

$$r_i \geq \sum_{j \neq i} \max_{\mathbf{p} \in \mathcal{P}_{\text{in}}} |B_{ij}(\mathbf{p})|$$

be a rigorous row-radius upper bound. Gershgorin's theorem and symmetry imply that every real eigenvalue λ of every admissible $B(\mathbf{p})$ lies in one of the intervals

$$[d_i^- - r_i,\ d_i^+ + r_i].$$

Consequently the entire family satisfies

$$\lambda_L \leq \lambda_k(B(\mathbf{p})) \leq \lambda_U,$$

where

$$\lambda_L = \min_i (d_i^- - r_i), \qquad \lambda_U = \max_i (d_i^+ + r_i).$$

AutoIA also forms

$$E(\mathbf{p}) = B(\mathbf{p}) - I$$

and rigorously bounds a matrix norm by

$$\|E(\mathbf{p})\|_2 \leq \rho.$$

Because B is symmetric, this independently gives

$$1 - \rho \leq \lambda_k(B(\mathbf{p})) \leq 1 + \rho.$$

The two independent eigenvalue enclosures are intersected. This is useful because the near-identity norm theorem can be sharp locally while Gershgorin can still prove positivity on boxes for which $\rho \geq 1$.

If the resulting lower bound satisfies

$$\boxed{\lambda_L > 0},$$

then

$$\boxed{A(\mathbf{p}) \succ 0 \quad \text{for every } \mathbf{p} \in \mathcal{P}_{\text{in}}.}$$

If a sufficient bound is inconclusive, AutoIA may request physical-box refinement. Failure of one sufficient theorem is not interpreted as proof that the matrix is non-SPD.

14.3 Spectral log-determinant bounds

For an SPD matrix, the determinant is positive. Since

$$A = LBL^T,$$

we have

$$\log\det A = 2\sum_{i=1}^{n} \log L_{ii} + \log\det B.$$

The eigenvalue enclosure immediately yields

$$n\log\lambda_L \le \log\det B \le n\log\lambda_U.$$

Hence a broad rigorous interval is

$$\boxed{2\sum_i \log L_{ii} + n\log\lambda_L \le \log\det A(\mathbf{p}) \le 2\sum_i \log L_{ii} + n\log\lambda_U.}$$

Every logarithm and reduction in this statement is evaluated with outward rounding.

14.4 Dependency-preserving matrix-log series

The broad spectral interval can be tightened without discarding the shared physical dependence. Choose a positive stored binary64 scaling constant c and write

$$B(\mathbf{p}) = c\,[I + F(\mathbf{p})].$$

AutoIA chooses c from the rigorous positive eigenvalue enclosure so that

$$\|F(\mathbf{p})\|_2 \le \rho_s < 1.$$

For a symmetric matrix with spectral radius below one,

$$\log(I+F) = F - \frac{F^2}{2} + \frac{F^3}{3} - \cdots.$$

Thus

$$\log\det B = n\log c + \operatorname{tr}\left(F - \frac{F^2}{2} + \cdots + (-1)^{r+1}\frac{F^r}{r}\right) + R_r,$$

where r is the retained series order. The retained matrix powers are ordinary AutoIA expressions and therefore preserve dependence on $\mathbf{p}$. The omitted trace tail satisfies

$$|R_r| \le n\frac{\rho_s^{r+1}}{(r+1)(1-\rho_s)}.$$

This gives a dependency-bearing log-determinant enclosure plus a rigorously bounded scalar tail. The final range is intersected with the independent spectral log-determinant interval.

14.5 Inverse quadratic forms without entrywise inversion

A second ubiquitous quantity is

$$q(\mathbf{p}) = b(\mathbf{p})^T A(\mathbf{p})^{-1} b(\mathbf{p}),$$

where $b(\mathbf{p}) \in \mathbb{R}^n$ may itself depend on the same uncertain inputs. Define

$$z(\mathbf{p}) = L^{-1} b(\mathbf{p}).$$

Then

$$q(\mathbf{p}) = z(\mathbf{p})^T B(\mathbf{p})^{-1} z(\mathbf{p}).$$

The eigenvalue theorem gives the broad independent bounds

$$\boxed{\frac{\|z(\mathbf{p})\|_2^2}{\lambda_U} \le q(\mathbf{p}) \le \frac{\|z(\mathbf{p})\|_2^2}{\lambda_L},}$$

with rigorous interval bounds used for $\|z(\mathbf{p})\|_2^2$.

The same scaling used for the matrix logarithm gives

$$B^{-1} = \frac{1}{c}(I+F)^{-1} = \frac{1}{c}\left[I - F + F^2 - \cdots + (-F)^r\right] + \mathcal{R}_r.$$

The inverse-series operator tail satisfies

$$\|\mathcal{R}_r\|_2 \le \frac{1}{c}\frac{\rho_s^{r+1}}{1-\rho_s}.$$

Therefore

$$\left|z^T \mathcal{R}_r z\right| \le \frac{\|z\|_2^2}{c}\frac{\rho_s^{r+1}}{1-\rho_s}.$$

The retained expression

$$\frac{1}{c} z^T \left[I - F + F^2 - \cdots + (-F)^r\right] z$$

remains dependency-bearing in the original physical inputs. AutoIA may also attempt its independent general parameter-dependent linear-solve theorem and intersect the resulting range if that theorem succeeds. A failure of the optional solve theorem cannot invalidate the already proved SPD series result.

14.6 Worked two-by-two example

Consider

$$A(x) = \begin{bmatrix} 2 + 0.1x & 0.05x \\ 0.05x & 1.5 - 0.05x \end{bmatrix}, \qquad x \in [-0.2, 0.2],$$

and

$$b(x) = \begin{bmatrix} 1 + 0.2x \\ 0.5 - 0.1x \end{bmatrix}.$$

All four uncertain matrix entries share the same scalar x. Replacing them by four independent intervals would admit matrices that are not equal to $A(x)$ for any physical x. AutoIA instead retains the common x, proves the SPD family after constant preconditioning, and can certify

$$\log\det A(x), \qquad \det A(x), \qquad b(x)^T A(x)^{-1} b(x)$$

with one structured theorem portfolio. This is the general mechanism used by V1; no covariance, Gaussian-process, or application-specific formula is built into the matrix semantic.

15 Validated implicit nonlinear solves

Many engineering codes define an internal state implicitly rather than by an explicit formula. Let

$$F(y, p) = 0$$

be such a residual equation. Here p denotes the physical uncertain inputs, and y denotes the unknown state. For a scalar root, $y \in \mathbb{R}$; for a square system, $y \in \mathbb{R}^n$.

AutoIA deliberately separates the *numerical proposal* from the *mathematical certificate*. A recognized SciPy solver may supply a good binary64 candidate y_0, but SciPy convergence is never accepted as proof. This design was chosen because the theorem that matters to the engineering user is existence and enclosure of an exact real solution, not reproduction of every floating-point iteration of Brent's method or MINPACK.

15.1 Dependency-bearing implicit predictor

Let

$$a(p) = (a_1(p), \ldots, a_r(p))^T$$

collect the uncertain residual arguments actually reached by the residual. At a nominal representative a_0 and nominal solution y_0, define the proposal Jacobians

$$J_y = \left.\frac{\partial F}{\partial y}\right|_{(y_0, a_0)}, \qquad J_a = \left.\frac{\partial F}{\partial a}\right|_{(y_0, a_0)}.$$

For a vector state, $J_y \in \mathbb{R}^{n \times n}$ and $J_a \in \mathbb{R}^{n \times r}$. If the nominal J_y is invertible, the implicit-function sensitivity proposal is

$$S = -J_y^{-1} J_a.$$

AutoIA then forms

$$\boxed{\widehat{y}(p) = y_0 + S\big(a(p) - a_0\big).}$$

The important feature is that $a(p)$ remains an AutoIA value carrying its original-input dependence. The root is therefore not immediately replaced by an independent scalar interval.

This choice is essential downstream. For example, if

$$y(p)^2 = p$$

and a later line computes $y^2 - p$, a dependency-bearing predictor can preserve near cancellation; an independent interval for y would generally destroy it.

15.2 Bracketed scalar `root_scalar`

The current V1 semantic supports a finite constant bracket

$$[a, b], \qquad a < b,$$

and a bracketed SciPy method. AutoIA first proves a uniform endpoint sign change. If

$$F(a, p) \in A, \qquad F(b, p) \in B,$$

then A and B must have strict opposite signs for every admissible physical input.

AutoIA then proves uniqueness in the bracket by enclosing

$$F_y(y,p) = \frac{\partial F}{\partial y}(y,p)$$

over the bracket. If one bracket-wide enclosure is too loose, the root variable is subdivided. Every accepted subinterval must exclude zero and all accepted subintervals must have the same derivative sign. The endpoint sign change and strict monotonicity then imply exactly one root in $[a,b]$ for every physical input.

For local correction define

$$e = y - \widehat{y}(p)$$

and choose the proposal preconditioner

$$C = J_y^{-1}.$$

Consider

$$T(e,p) = e - CF(\widehat{y}(p) + e, p).$$

AutoIA proves

$$|CF(\widehat{y}(p), p)| \le c$$

and, for $e \in [-d,d]$,

$$|1 - CF_y(\widehat{y}(p) + e, p)| \le \rho.$$

If

$$\rho < 1, \qquad \frac{c}{1-\rho} \le d,$$

then Banach's contraction theorem proves one correction in the tube and hence

$$\boxed{y(p) \in \widehat{y}(p) + [-d,d].}$$

Because the bracket root was already proved unique, this local correction is the same mathematical root requested by the bracketed call.

Example 1: square root as an implicit state. Let $p \in [1, 1.21]$ and

$$F(y,p) = y^2 - p, \qquad y \in [0.5, 2].$$

The positive branch has exact solution $y = \sqrt{p}$. A nominal candidate near $p_0 = 1.105$ gives a sensitivity proposal

$$\frac{dy}{dp} \approx \frac{1}{2y_0}.$$

AutoIA retains this first-order dependence and proves the remaining correction.

Example 2: Colebrook–White friction factor. The Darcy friction factor f is defined by

$$\frac{1}{\sqrt{f}} + 2\log_{10}\left(\frac{\epsilon/D}{3.7} + \frac{2.51}{Re\sqrt{f}}\right) = 0.$$

Here Re is Reynolds number and ϵ/D is relative roughness. The canonical V1 example uses the user's ordinary `root_scalar` call, then AutoIA independently proves the bracket semantics and the dependency-bearing correction.

15.3 Square-system `fsolve`

For

$$F(y,p)=0, \qquad y\in\mathbb{R}^n,$$

the V1 `fsolve` theorem is intentionally local because `fsolve` itself is an initial-guess local solver. With a finite proposal $C\approx J_y^{-1}$, AutoIA proves

$$\|CF(\widehat{y}(p),p)\|_\infty \le c$$

and, over $\|e\|_\infty\le d$, a componentwise defect matrix

$$B \ge |I-CJ_y(\widehat{y}(p)+e,p)|.$$

Let

$$\rho=\|B\|_\infty.$$

If

$$\rho<1, \qquad \frac{c}{1-\rho}\le d,$$

then a unique solution exists in that correction tube for every physical input. This theorem does not claim that no remote solution exists elsewhere.

Example 3: coupled two-state equilibrium. For uncertain a,b, consider

$$F_1(y,a,b)=y_1+0.2y_2-a,$$

$$F_2(y,a,b)=0.1y_1+y_2+0.05y_1y_2-b.$$

The nominal solve proposes y_0, the implicit Jacobian constructs a predictor, and the contraction proof validates a local unique branch.

15.4 Derivative trust boundary

Forward-mode differentiation is used to construct nominal and proof expressions. A nominal derivative is only proposal information. Proof-critical derivative ranges are themselves validated through the supported scalar language and trusted range machinery. A derivative through an unsupported operation causes an explicit failure rather than a floating-point finite-difference substitute.

16 Generator-aware semantic linear solve

Suppose the solve inputs contain shared auxiliary generators:

$$A(\mathbf{q},\boldsymbol{\varepsilon})\in\widehat{A}(\mathbf{q})+\sum_{j=1}^{s}A_j\varepsilon_j+R_A,$$

$$b(\mathbf{q},\boldsymbol{\varepsilon})\in\widehat{b}(\mathbf{q})+\sum_{j=1}^{s}b_j\varepsilon_j+R_b.$$

AutoIA proposes a solution of the form

$$\boxed{X(\mathbf{q},\boldsymbol{\varepsilon})=P(\mathbf{q})+\sum_{j=1}^{s}u_j\varepsilon_j.}$$

For generator ε_j, a useful sensitivity proposal is obtained from a reference matrix A_{ref} by

$$A_{\mathrm{ref}} u_j = b_j - A_j x_0,$$

where $x_0 = P(0)$. Again, this is only a proposal.

The proof then evaluates the actual residual

$$r(\mathbf{q}, \boldsymbol{\varepsilon}) = b(\mathbf{q}, \boldsymbol{\varepsilon}) - A(\mathbf{q}, \boldsymbol{\varepsilon}) X(\mathbf{q}, \boldsymbol{\varepsilon})$$

over the joint domain

$$\mathbf{q} \in \mathcal{Q}, \qquad \boldsymbol{\varepsilon} \in [-1, 1]^s,$$

and uses the same fixed-point theorem. Incoming generator identifiers are preserved in the explicit predictor when justified. The remaining componentwise correction proven by the contraction theorem is assigned fresh independent auxiliary generators because the theorem proves only a box for the residual error, not a stronger coupled geometry.

17 Path-aware execution and uncertain control flow

Consider a scalar predicate such as

`lhs < rhs`.

AutoIA forms the validated difference

$$d = \texttt{lhs} - \texttt{rhs}$$

and computes a rigorous range

$$d \in [\underline{d}, \overline{d}].$$

For the strict less-than relation:

- if $\overline{d} < 0$, the predicate is proved true throughout the current box;
- if $\underline{d} \geq 0$, it is proved false throughout the current box;
- otherwise the predicate is unresolved.

Analogous exact-real rules are used for $\leq, >, \geq, ==, != $ and scalar truth checking.

If the predicate is unresolved, AutoIA may split the physical input box and re-execute. For an exact affine one-variable predicate

$$d(p_i) = a + b p_i, \qquad b \neq 0,$$

the exact root

$$p_i^\star = -a/b$$

can be used as a direct partition point when it is rigorously recognized.

For a general nonlinear or multivariate decision surface, finitely many Cartesian boxes may not align with the exact path boundary. If subdivision cannot resolve the predicate under the allowed budget, AutoIA conservatively enumerates both feasible Boolean outcomes and certifies the continuation of each path separately. If the two branch enclosures are Y_T and Y_F, then

$$\mathcal{P}(\mathcal{P}_{\mathrm{in}}) \subseteq Y_T \cup Y_F.$$

The branch formula may be evaluated on a superset of its true feasible domain; this can make the result wider or cause a boundary singularity to fail certification, but it cannot omit a true output.

Path-local continuation is essential. AutoIA does not immediately hull two branch values and then continue the rest of the program. Instead every feasible path carries its own validated values through downstream arithmetic, elementary functions, and semantic solves. The union is formed only at the final piecewise result.

18 Exact closed-endpoint comparison metadata

Outward rounding is required for rigorous numerical enclosures, but it can make an expression that is mathematically exact at a closed endpoint appear to cross a comparison threshold by a few ulps. For example, if

$$c \in [0.95, 1]$$

then the physical statement $c > 1$ is false everywhere, even if a rounded polynomial range of $c - 1$ is

$$[-0.05, 7 \times 10^{-18}].$$

To preserve exact Python comparison semantics in such cases, AutoIA carries a *proof-only exact affine shadow* whenever an expression remains exactly affine in the original physical inputs. Write

$$s(p) = \beta_0 + \sum_{i=1}^{m} \beta_i p_i,$$

where each β_i is stored as an exact rational number equal to the exact real value of the corresponding binary64 constant. Over a box, the exact range of this affine function is obtained by choosing each interval endpoint according to the sign of β_i. This metadata is used only to decide comparisons; it never replaces the trusted numerical enclosure used for arithmetic.

The shadow survives addition, subtraction, negation, multiplication by an exact constant, and division by an exact nonzero constant. Nonlinear operations discard it. This design was chosen because it solves endpoint semantics without weakening outward rounding or introducing symbolic algebra into general nonlinear arithmetic.

Example 1. For $x \in [0.95, 1]$, the branch

```
if x > 1.0:
    raise ValueError(...)
```

is proved infeasible exactly.

Example 2. For the same box, the predicates $x < 1$ and $x == 1$ define two feasible path pieces. AutoIA retains the equality boundary rather than silently assigning it to the wrong strict branch.

19 Proof-driven physical-domain refinement

A broad enclosure may fail to prove a domain condition even when the original engineering model is valid throughout the physical box. AutoIA therefore treats several domain obligations as requests for refinement rather than immediate terminal failures.

For a reciprocal

$$z = \frac{1}{d(p)},$$

the proof obligation is

$$0 \notin d(\mathcal{P}_{\text{in}}).$$

If the current enclosure of d contains zero but does not prove an actual zero, the controller may subdivide the physical box, re-execute the model, and prove the denominator nonzero on each child. The same architectural pattern is used for supported elementary-domain conditions such as positivity of a logarithm argument.

This is sound because no parent proof is accepted. Each retained child is independently certified and the child boxes cover the parent domain.

Example 1: removable over-width in a denominator. A denominator may be positive on the true domain while dependency loss in a broad intermediate enclosure crosses zero. Two physical subboxes can recover the positive proof without changing the engineering formula.

Example 2: shell-and-tube heat exchanger algebra. A public effectiveness formula contains nested reciprocals. On a broad (NTU, C_r) box, a whole-box denominator enclosure can cross zero even though finer physical boxes certify. The V1 controller now requests reciprocal-driven subdivision before failing closed.

This mechanism does not perform symbolic cancellation of removable singularities. If a branch formula genuinely approaches a $0/0$ form on the closure of a strict path, generic interval division can still fail. That is a separate mathematical issue.

20 Fixed-data regular-grid multilinear interpolation

The V1 semantic for `scipy.interpolate.RegularGridInterpolator` targets fixed numeric rectilinear grids and fixed numeric table values with `method="linear"`. Uncertainty is allowed in the query coordinates. This subset was chosen because tabulated maps are ubiquitous in engineering and piecewise multilinear interpolation has a transparent exact algebraic meaning.

Let there be d grid dimensions. On dimension i, let the strictly monotone grid coordinates be

$$g_{i,0}, g_{i,1}, \ldots, g_{i,n_i-1}.$$

After internal normalization to ascending order, suppose a query coordinate x_i lies in cell

$$g_{i,j_i} \le x_i \le g_{i,j_i+1}.$$

Define the normalized local coordinate

$$t_i = \frac{x_i - g_{i,j_i}}{g_{i,j_i+1} - g_{i,j_i}}.$$

For a scalar table, the multilinear interpolant in that cell is

$$\boxed{I(x) = \sum_{b \in \{0,1\}^d} v_{j_1+b_1,\ldots,j_d+b_d} \prod_{i=1}^{d} \big[b_i t_i + (1-b_i)(1-t_i)\big].}$$

The vector $b = (b_1, \ldots, b_d)$ selects one of the 2^d cell corners. Because the tabulated values are fixed constants, this is ordinary AutoIA arithmetic in the uncertain query coordinates and preserves their dependence.

If a query box crosses a grid boundary, the boundary comparison becomes an ordinary AutoIA path decision. Every feasible cell formula is certified on its path and the final piecewise hull covers all cells reached by the physical input box. No global min/max lookup is substituted for the interpolant.

Example 1: bilinear table. For a two-dimensional cell with corner values $v_{00}, v_{10}, v_{01}, v_{11}$,

$$I(x_1, x_2) = (1 - t_1)(1 - t_2)v_{00} + t_1(1 - t_2)v_{10} + (1 - t_1)t_2v_{01} + t_1t_2v_{11}.$$

If the uncertain box remains inside the cell, this one expression is propagated through downstream code.

Example 2: aerodynamic lookup crossing cells. Suppose Mach number and angle of attack are uncertain and both cross table breakpoints. AutoIA splits/enumerates the corresponding cell paths, evaluates the proper bilinear formula on each, and hulls only at the final path result.

Example 3: out-of-domain semantics. With `bounds_error=True`, a physical box containing an out-of-grid query means the user model is not defined everywhere and certification fails. With `bounds_error=False` and a finite fixed `fill_value`, the outside path returns that constant. With `fill_value=None`, the first or last cell formula supplies the same linear extrapolation semantics as SciPy.

Nonlinear interpolation methods, uncertain grid coordinates, and uncertain table values are deliberately not part of this V1 semantic.

21 Adaptive subdivision controller

Subdivision is a sharpness/resource strategy, not a correctness prerequisite. Every retained child box is independently certified. If a controller chooses a poor direction, the result may be slower or wider but does not become unsound.

The practical split score introduced in version 0.19.0.dev1 uses the already certified parent polynomial. For candidate coordinate q_d, a half-box child has a shifted center

$$s_d = \pm h_d/2$$

and new coordinate r_d satisfying

$$q_d = r_d + s_d, \qquad r_d \in [-h_d/2, h_d/2].$$

The binomial identity

$$q_d^{\alpha_d} = \sum_{b=0}^{\alpha_d} \binom{\alpha_d}{b} s_d^{\alpha_d - b} r_d^b$$

permits a cheap predicted re-centering without executing the engineering model. The score estimates derivative-sign resolution, reduced-Bernstein burden, and output-width gain. Only the selected child pair is actually certified. The controller then checks realized structural/width improvement before accepting the split.

The old expensive one-step lookahead remains available as a research oracle. Because controller decisions are outside the trusted proof kernel, agreement with that oracle is an efficiency/sharpness validation issue rather than a soundness condition.

22 Automatic configuration as a portfolio of independent proofs

The novice API

```
autoia(model, box)
```

starts from a release baseline consisting of the polynomial representation at order four. The integer $k = 4$ here is the maximum retained total polynomial degree. The controller may make a bounded number of optional retries when the baseline exposes one of two generic structural mechanisms:

1. the rigorous remainder dominates the retained physical-polynomial variation, suggesting that a modestly higher order may help; or
2. approximation-error ancestry is reused/recombined downstream, suggesting that shared generators may preserve cancellation.

These triggers are heuristics for cost and sharpness only.

Suppose successful independent runs produce output intervals

$$Y^{(1)}, Y^{(2)}, \ldots, Y^{(k)}$$

for the same scalar output. Because every interval contains the exact output, the intersection

$$\boxed{Y = \bigcap_{j=1}^{k} Y^{(j)}}$$

also contains the exact output. Componentwise, the reported bounds are

$$\underline{Y} = \max_j \underline{Y}^{(j)}, \qquad \overline{Y} = \min_j \overline{Y}^{(j)}.$$

Thus the selector is not trusted proof arithmetic. A poor selector can waste work or miss an optional tightening; it cannot make the intersection unsound.

Example. If a polynomial/order-4 run proves $[-2, 5]$ and a generator/order-4 retry proves $[-1, 6]$, the intersection is $[-1, 5]$, which is rigorous even though neither run dominates the other completely.

23 Application-theorem interface

AutoIA's generic execution engine should not absorb application-specific mathematics. Instead, an application expert may state a theorem externally and use a small rigorous context to evaluate its numerical consequences.

Let a theorem prove that a scalar target f is nondecreasing in a certified intermediate z on the current domain. If AutoIA encloses

$$z(p) \in [\underline{z}, \overline{z}],$$

then the theorem may conclude

$$f(p) \le g(\overline{z})$$

for the specified monotone outer function g. The application layer records which assumptions were checked and returns a typed rigorous upper or lower bound.

Independent application theorems can also be combined. If

$$f(p) \le U_1 \qquad \text{and} \qquad f(p) \le U_2,$$

then

$$f(p) \le \min(U_1, U_2).$$

Similarly, independent lower bounds combine by a maximum, and two full certified intervals for the same quantity combine by intersection.

Example 1: monotonicity. If $f(x) = e^x$ and $x \in [\ell, u]$, monotonicity gives

$$f(x) \le e^u.$$

The theorem is supplied by the application expert; the exponential evaluation is performed rigorously by AutoIA.

Example 2: Lipschitz theorem. If the expert has proved

$$|f(x) - f(c)| \le L\|x - c\|_2$$

on the application domain, AutoIA can rigorously evaluate $f(c)$, the box radius, and the resulting upper/lower bound without embedding the theorem in the core engine.

This separation is the permanent architectural boundary used by application packages such as downstream GP application: application mathematics may depend on AutoIA, while AutoIA itself remains application-independent.

24 Prepared sessions and repeated-query certification

Version 1.0.2 retains the V1 separation of static program preparation from repeated box certification. Let one fixed engineering program $\mathcal{P}$ and one fixed AutoIA configuration be used for a sequence of boxes

$$\mathcal{P}_{\text{in}}^{(1)}, \mathcal{P}_{\text{in}}^{(2)}, \ldots.$$

A prepared session reuses the local semantic-dispatch clone and one audited trusted backend. More importantly, it stores previously certified boxes and results in a bounded certificate cache.

24.1 Exact repeated-box reuse

If a new query box has exactly the same stored binary64 endpoints as a cached box, the previous certificate already proves the same statement. No engineering model execution is required.

24.2 Restriction of a parent certificate to an interior subbox

Suppose a parent box D has already been certified and a new query box Q satisfies

$$Q \subseteq D.$$

Set-theoretically, every certificate valid on D is automatically valid on Q. For a polynomial certificate, AutoIA can do better than returning the same parent output interval: it re-centers and re-ranges the stored polynomial on Q.

Let the old centered coordinate be

$$\mathbf{q} = \mathbf{p} - \mathbf{c}_{\text{old}},$$

and the new centered coordinate be

$$\mathbf{r} = \mathbf{p} - \mathbf{c}_{\text{new}}.$$

Then exactly

$$\mathbf{q} = \mathbf{r} + \mathbf{s},$$

where

$$\mathbf{s} = \mathbf{c}_{\text{new}} - \mathbf{c}_{\text{old}}.$$

For a monomial

$$\mathbf{q}^{\alpha} = \prod_{i=1}^{m} (r_i + s_i)^{\alpha_i}.$$

Using the multivariate binomial expansion,

$$\mathbf{q}^{\alpha} = \sum_{\beta \le \alpha} \left[\prod_{i=1}^{m} \binom{\alpha_i}{\beta_i} s_i^{\alpha_i - \beta_i} \right] \mathbf{r}^{\beta}.$$

Therefore if

$$P(\mathbf{q}) = \sum_{\alpha} c_{\alpha} \mathbf{q}^{\alpha},$$

the exact re-centered coefficient of $\mathbf{r}^{\beta}$ is

$$c'_{\beta} = \sum_{\alpha \ge \beta} c_{\alpha} \prod_{i=1}^{m} \binom{\alpha_i}{\beta_i} s_i^{\alpha_i - \beta_i}.$$

The exact shift s_i is not assumed to equal the ordinary floating-point subtraction of the two stored centers. The trusted backend forms an interval S_i containing the exact real difference. Every transformed coefficient is computed as a rigorous interval using S_i, while an ordinary binary64 point coefficient is stored as a proposal. The difference between the exact coefficient interval and the stored proposal is rigorously ranged over the new centered box and accumulated into an extra remainder.

Hence the re-centered value remains a valid certificate on the new domain. Shared-generator coefficients are copied unchanged because the coordinate shift acts on the physical polynomial P, not on the auxiliary variables $\boldsymbol{\varepsilon}$.

24.3 Containment check for the actual centered query domain

The user query Q is not quite the domain used by the polynomial executor. The executor constructs an outward symmetric centered envelope around each new binary64 center. Let that actual represented domain be $\widetilde{Q}$. Certificate restriction is used only when AutoIA proves

$$\widetilde{Q} \subseteq D.$$

This prevents the known one-ulp symmetric-centering enlargement from causing a parent certificate to be applied outside the domain on which it was proved. If this containment check fails, AutoIA performs a fresh certification instead.

24.4 Many-box execution and multicore CPU proof

For a collection of N independent physical boxes, prepared execution supports serial, threaded, and process-parallel modes. With `workers="auto"` and `parallel="auto"`, AutoIA resolves an available CPU worker count and prefers process parallelism when the model can be serialized. Optional `cloudpickle`, when installed, permits many local functions and closures to be serialized for spawn-based workers; it is execution infrastructure and is not part of the proof.

In process mode each worker constructs one prepared session and one independently audited trusted backend, then handles multiple boxes. The mathematical statement proved for box B_k is independent of which worker evaluated it. Parallel scheduling therefore changes wall time but not the inclusion theorem. The public many-box call records non-proof execution metadata such as the resolved worker count, serial/thread/process mode, chunk size, elapsed time, and model serializer.

A streaming interface keeps only a bounded number of chunks in flight, so the user does not need to store all N boxes or all N full result objects in memory simultaneously. This is execution infrastructure for large repeated-query workloads; it does not decide which boxes an optimization or reliability algorithm should generate.

24.5 MPS and CUDA proposal acceleration with CPU verification

AutoIA V1 also exposes optional Torch-backed proposal helpers for Apple Metal through the PyTorch MPS device and for NVIDIA hardware through CUDA. These accelerator calculations are explicitly outside the trusted proof base. A GPU may evaluate many center points, ranking scores, nominal preconditioners, or other proposal quantities, but the resulting object carries `machine_rigorous=False`. Any final AutoIA certificate is recomputed or verified independently by the trusted CPU binary64 backend.

The V1 trust boundary is therefore

optional MPS/CUDA proposal or assist $\longrightarrow$ trusted multicore CPU verification.

This allows applications to exploit available accelerators without making the correctness of an enclosure depend on GPU rounding modes, tensor-library kernel choices, or hardware-specific reduced precision. A future fully trusted GPU backend would require a separate directed-rounding implementation and audit.

25 Program-level soundness theorem

We can summarize the architecture as a compositional theorem.

Assumptions. Assume:

1. the trusted binary64 backend's audited machine assumptions hold;
2. every proof-critical interval operation returns an enclosure of its exact real counterpart;
3. every supported validated scalar/semantic operation satisfies its enclosure contract;
4. every representation conversion is inclusion-preserving;
5. every retained path/subdivision child is certified over a domain whose union covers the relevant user input domain;

6. every semantic linear or nonlinear solve accepted by the system satisfies its stated residual/contraction and, where applicable, bracket-uniqueness theorem;

7. every accepted SPD matrix-family operation satisfies its structural symmetry, positive-eigenvalue, matrix-series, and tail-bound proof obligations;

8. every accepted fixed-data interpolation path uses the correct multilinear cell formula on a domain covering that path; and

9. the program remains within the documented supported-language semantics.

Conclusion. For every user input $\mathbf{p} \in \mathcal{P}_{\text{in}}$, the exact real output of the supported program semantics belongs to the finite union of the validated output sets returned by AutoIA. Consequently the reported global lower/upper arrays contain every exact scalar output.

Proof sketch. The proof is by induction through execution. Initial uncertain inputs are represented by enclosures containing every physical value in the input box. Every supported operation maps valid input enclosures to an output enclosure containing the exact operation on all admissible inputs. Semantic solves are validated independently by their residual/contraction theorem. If execution branches, the path engine retains every feasible outcome or subdivides the physical domain into certified children whose union covers the original domain. Thus the invariant holds on every active path. Final range methods enclose each representation's concretization, and the finite union/hull of path results contains the exact output set.

Prepared-session restriction does not alter the theorem: a parent certificate is reused only on a subdomain proved to lie inside the parent's certified domain, and the coordinate transform itself is rigorously enclosed.

26 What ordinary floating-point calculations are allowed to do

A recurring AutoIA design principle is that an approximation may be poor without making the final result unsound. Ordinary floating point is therefore acceptable for:

- polynomial coefficient proposals;
- nominal/reference linear solves;
- approximate inverses and preconditioners;
- nominal Cholesky factors and constant SPD preconditioners;
- midpoint representatives of interval matrix families;
- branch split scores and controller heuristics;
- candidate Taylor centers;
- integer argument-reduction proposals for trigonometric functions;
- cache-choice heuristics such as choosing the smallest containing parent certificate.

Such calculations may affect sharpness, performance, or whether a proof attempt succeeds. They must not be used as the final reason an inclusion is accepted. The proof always returns to rigorous residual, interval, range, or domain checks.

27 V1 heterogeneous execution policy

Execution scheduling is outside the mathematical proof. For the preferred finite many-box interface, the default `workers="auto"`, `parallel="auto"` policy stays serial for fewer than four chunks and otherwise uses at most one CPU worker per chunk, capped by the available CPU count. Explicit user scheduling overrides the heuristic. The resulting certificates are unchanged in meaning because every box is independently verified by the trusted CPU binary64 backend.

Optional MPS and CUDA helpers are proposal/assist devices only. Version 1 deliberately resolves `device="auto"` to CPU rather than guessing GPU profitability from batch size. The application must explicitly request `mps` or `cuda`. This is a performance policy and does not alter the trust boundary: GPU results are never machine-rigorous certificates.

28 Important V1 limitations and explicit non-guarantees

AutoIA is a validated-execution system for a deliberately bounded Python/NumPy/ SciPy language surface. The following are current non-claims.

1. **Exact hulls are not promised.** Every accepted interval is an inclusion certificate, not necessarily the tightest possible interval.

2. **Arbitrary Python is not supported.** General side effects, uncertainty-dependent indexing, arbitrary Boolean masks, unrestricted uncertainty-dependent loop termination, and opaque compiled-extension internals are outside the frozen V1 language unless a separate semantic primitive exists.

3. **Local imports are a dispatch boundary.** The source-level shadow namespace prepares visible globals and reached source-backed helpers; imports performed dynamically inside a function body are not generally rewritten.

4. **Shared generators are affine auxiliary variables.** Products such as $q_i\varepsilon_j$ and $\varepsilon_i\varepsilon_j$ are bounded back into new rigorous error rather than retained as a richer nonlinear auxiliary basis.

5. **Generator compression is not yet general.** Long calculations can accumulate auxiliary generator identifiers.

6. **Implicit solves are conditional semantics.** Bracketed `root_scalar` requires a uniform bracket/uniqueness proof; `fsolve` proves only a local unique branch around the validated tube. Bifurcations, singular Jacobians, or merged branches can fail closed.

7. **Interpolation support is intentionally narrow.** V1 certifies fixed-data `RegularGrid Interpolator` queries with `method="linear"`. Nonlinear methods and uncertain table/grid data are not claimed.

8. **General eigensystems are not yet certified.** `scipy.linalg.eigh` with AutoIA-dependent matrices is recognized and fails explicitly with `UnsupportedOperationError`; no eigenvalue or eigenvector certificate is produced.

9. **General ODE/DAE and optimizer semantics are not yet certified.** Calls such as `solve_ivp` or arbitrary constrained optimization remain outside the V1 semantic set unless a future version adds a theorem-backed primitive.

10. **Removable singularities are not symbolically regularized.** A branch formula whose closure contains a true $0/0$ form may fail even if the piecewise mathematical function has a finite limit.

11. **GPU proof execution is not part of V1.** V1 exposes optional PyTorch MPS and CUDA proposal/assist helpers, but these results are explicitly non-rigorous. The current trusted proof kernel, including parallel many-box verification, is CPU binary64.

12. **Piecewise cached restriction and giant many-box kernels remain limited.** Prepared sessions provide exact and contained-box reuse, but not a fully general symbolic restriction engine for arbitrary piecewise branch geometry or one dense GPU-sized batch proof kernel.

A recognized unsupported operation must fail closed. The desired V1 behavior is therefore not "every engineering program certifies," but rather "supported semantics certify rigorously, and unsupported or unproved semantics are made explicit without silently returning an uncertified result."

29 A compact end-to-end mathematical example

Consider a two-parameter model

$$p_1 \in [0.9, 1.1], \qquad p_2 \in [-0.1, 0.1],$$

with

$$A(\mathbf{p}) = \begin{bmatrix} 2 + p_1^2 & 0.2 \\ -0.1p_2 & 1.5 + 0.05p_1 \end{bmatrix},$$

$$b(\mathbf{p}) = \begin{bmatrix} 1 + 0.1p_2 \\ 0.5 + 0.05p_1^2 \end{bmatrix},$$

and let $u(\mathbf{p})$ solve

$$A(\mathbf{p})u(\mathbf{p}) = b(\mathbf{p}).$$

Suppose the program returns

$$y_1 = u_1, \qquad y_2 = e^{u_2}, \qquad y_3 = u_1 - u_1.$$

AutoIA first constructs rigorous centered coordinates for p_1, p_2. Polynomial arithmetic retains the quadratic occurrences of p_1 up to the chosen order. The semantic solve constructs a polynomial (and, in generator mode, auxiliary sensitivity) predictor, then proves a contraction bound on the complete matrix family. The exponential is composed by the validated Taylor machinery. Shared generator provenance can preserve exact downstream cancellation in y_3, so y_3 can certify as zero even when intermediate approximations carry error.

If a prepared session first certifies the master box and later receives a smaller interior box, the stored output polynomial certificates may be re-centered using the exact binomial coordinate transform with rigorous coefficient-error capture. The model and the semantic solve need not run again if the new centered domain is proved to lie inside the parent domain.

30 Additional worked examples connecting the concepts

30.1 Example A: polynomial truncation versus interval dependence loss

Let

$$f(x) = x - x + x^2, \qquad x \in [-1, 1].$$

The exact function is $x^2 \in [0, 1]$. A dependency-blind interval calculation of the first two terms could form

$$[-1, 1] - [-1, 1] = [-2, 2]$$

before adding $[0, 1]$. A physical-input polynomial representation keeps the same symbolic coordinate q in both occurrences and cancels the linear terms before final ranging.

30.2 Example B: generator cancellation after approximation

Suppose an elementary approximation produces

$$y = P(q) + g\varepsilon + R, \qquad \varepsilon \in [-1, 1].$$

If downstream code returns $y - y$, shared use of the same generator identifier cancels $g\varepsilon$ exactly. Replacing both occurrences by independent interval remainders would not preserve that fact.

30.3 Example C: path-aware absolute-value-like code

Consider

```
if x >= 0:
    y = x
else:
    y = -x
return y + x*x
```

for $x \in [-1, 1]$. AutoIA retains two path-local expressions and ranges them separately. It does not first hull y to $[0, 1]$ and then forget which sign of x produced it.

30.4 Example D: semantic linear solve

For

$$A(k) = \begin{bmatrix} 2k & -k \\ -k & k \end{bmatrix}, \qquad b(F) = \begin{bmatrix} F \\ 0 \end{bmatrix},$$

with uncertain stiffness k and load F, NumPy's nominal `linalg.solve` is only a proposal. AutoIA verifies the matrix family by a preconditioned residual/contraction theorem and returns a dependency-bearing solution enclosure.

30.5 Example E: implicit solve followed by dependent arithmetic

Let $y(p)$ be the positive solution of $y^2 = p$. If downstream code computes

$$r = y^2 - p,$$

the exact value is zero. A root interval alone loses the relation between y and p; the implicit predictor $\widehat{y}(p) + [-d, d]$ preserves enough shared dependence to make the residual enclosure far smaller.

30.6 Example F: interpolation across a grid boundary

Let a one-dimensional table have nodes $g_0 < g_1 < g_2$, and suppose $x \in [g_0 + \delta, g_2 - \delta]$ crosses g_1. The interpolant is one affine formula on $[g_0, g_1]$ and another on $[g_1, g_2]$. AutoIA treats $x \le g_1$ as a path boundary, certifies each affine continuation, and hulls the two final outputs. This is exact with respect to the piecewise-linear interpolation semantics apart from outward arithmetic rounding.

30.7 Example G: independent theorem portfolio

Suppose a generic execution proves $f \in [-3, 7]$, a monotonicity theorem proves $f \le 5$, and a Lipschitz theorem proves $f \ge -2$. Combining independent certificates gives

$$f \in [-2, 5].$$

No theorem needs to know how the others were obtained.

30.8 Example H: correct fail-closed behavior for modal analysis

For a structural problem

$$K(p)\phi = \lambda M(p)\phi,$$

V1 recognizes `scipy.linalg.eigh` but does not yet possess the spectral separation/eigenspace theorem needed for a general certificate. The correct V1 result is therefore an explicit unsupported-operation error, not a nominal frequency with a misleading "rigorous" label.

30.9 Example I: why an SPD matrix family is not an entrywise interval matrix

Let

$$A(t) = \begin{bmatrix} 1+t & t \\ t & 1+t \end{bmatrix}, \qquad t \in [0, 0.4].$$

The same scalar t appears in all four uncertain locations. Entrywise intervalization would admit, for example, a matrix with one off-diagonal entry zero and the other equal to 0.4, although no physical value of t creates that matrix. The structured matrix-family representation excludes such fictitious combinations and permits preconditioned SPD reasoning over the actual one-parameter family.

30.10 Example J: inverse quadratic form by a dependency-bearing series

Suppose a preconditioned two-by-two family has

$$B(s) = c[I + F(s)], \qquad \|F(s)\|_2 \le 0.2.$$

Retaining terms through order two gives

$$B(s)^{-1} \approx \frac{1}{c}[I - F(s) + F(s)^2].$$

The omitted operator norm is no larger than

$$\frac{1}{c}\frac{0.2^3}{1 - 0.2} = \frac{0.01}{c}.$$

For $z(s) = L^{-1}b(s)$, the retained quadratic expression

$$\frac{1}{c}z(s)^T[I - F(s) + F(s)^2]z(s)$$

still contains the original uncertain variable s; only the remaining tail is collapsed to a scalar rigorous error interval.

30.11 Example K: parallel boxes do not create a parallel-proof theorem

Let three boxes B_1, B_2, B_3 be certified on three CPU worker processes. If worker k proves

$$f(B_k) \subseteq Y_k,$$

then the collection $\{Y_1, Y_2, Y_3\}$ is valid regardless of execution order. No theorem combines floating-point states across workers. Parallelism is only a scheduler around three independent proof calls.

30.12 Example L: GPU proposal versus CPU proof

Suppose an MPS kernel evaluates a box center and returns the float32 proposal

$$\widehat{f}(c) = 1.2374.$$

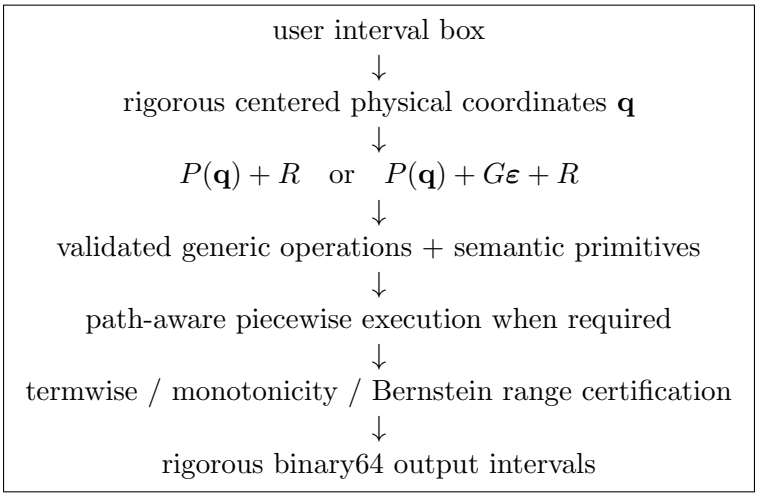
This number is useful for ranking or initialization but proves nothing. AutoIA then independently certifies the full physical box on CPU and obtains

$$f(B) \subseteq [1.18, 1.31].$$

Only the latter interval is a certificate. A wrong GPU proposal may reduce performance, but it cannot invalidate the CPU inclusion proof.

31 Summary of the current mathematical architecture

The current architecture can be summarized by the chain



with a prepared-session layer that can reuse the static dispatch architecture and, more importantly, restrict already-proved certificates to eligible interior query boxes.

The central theoretical thesis is that sharp automatic interval analysis for engineering code is best treated as a structured validated-execution problem: retain original-input dependence explicitly, introduce auxiliary variables only when their provenance is proved, treat important numerical operations according to their mathematical semantics, and confine correctness-critical floating-point work to a small audited enclosure kernel. Performance strategies are then free to be aggressive because any proposed approximation must still pass an independent rigorous certificate before it is accepted.

32 Transparent fitted exact-GP posterior semantics in Version 1.0.2

Let $x \in \mathbb{R}^d$ denote an uncertain query vector with d input coordinates. For a supported fitted scalar exact Gaussian process, AutoIA freezes the numerical fitted state and reconstructs the posterior using validated arithmetic in the uncertain coordinates.

Let $x_i \in \mathbb{R}^d$, for $i = 1, \ldots, n$, denote the n frozen training inputs. Let $k(x, x_i)$ denote the supported stationary covariance kernel, and define the kernel vector

$$k_x = \big(k(x, x_1), \ldots, k(x, x_n)\big)^T.$$

Let $\alpha \in \mathbb{R}^n$ denote the frozen posterior coefficient vector, $L \in \mathbb{R}^{n \times n}$ the frozen nonsingular lower-triangular Cholesky factor, and $m \in \mathbb{R}$ the frozen scalar mean. The internal latent posterior mean is

$$\mu_{\text{int}}(x) = m + k_x^T \alpha.$$

Define $v(x) \in \mathbb{R}^n$ as the unique solution of

$$Lv(x) = k_x.$$

The internal latent posterior variance is

$$s^2_{\text{int}}(x) = k(x, x) - v(x)^T v(x).$$

All occurrences of the uncertain vector x in these expressions are evaluated with AutoIA validated values. The frozen quantities x_i, α, L, and m are treated as stored binary64 data.

If the fitted model uses an affine output standardization, let $a \in \mathbb{R}$ be its stored offset and let $b > 0$ be its stored scale. The user-coordinate posterior is

$$\mu(x) = a + b\,\mu_{\text{int}}(x), \qquad s^2(x) = b^2 s^2_{\text{int}}(x).$$

The Version 1.0.2 semantic recognizer supports scalar exact GPs with RBF or Matern covariance ($\nu \in \{1/2, 3/2, 5/2\}$), an optional positive output-scale kernel wrapper, zero or scalar constant mean, Gaussian observation noise, and the documented affine input/output transforms. Recognition is deliberately fail-closed. AutoIA does not claim a general validated Torch execution model. The statistical fit, autograd, training convergence, and accelerator arithmetic are not certified; the proof statement is conditional on the frozen stored binary64 fitted state. An independent 80-digit audit checks 504 posterior mean/variance containment witnesses across the four supported kernel profiles.